

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A Python nyelv beépített adattípusai

Szöveg	str	Egy karaktersorozat ábrázolására való.
Szám	int, float, complex	Egész, valós és komplex számok ábrázolására való típusok.
Sorozat	list, tuple, range	Értékek sorozatának ábrázolására való típusok.
Leképezés	dict	Kulcs-érték párok ábrázolására való típus.
Halmaz	set, frozenset	Értékek halmazának ábrázolására való típusok.
Logikai	bool	Logikai értékek (True és False) ábrázolására való típus.
Bináris	bytes, bytearray, memoryview	Bináris adatok ábrázolására való típusok.
Semmi	NoneType	A None érték, azaz a semmi típusa.
		Nincs önálló karakter (char) és bájt (byte) típus!

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - További Python típusok**
 - Python function adattípus
 - 23 Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A NoneType adattípus

Python VIRTUÁLIS adattípus

- A `NoneType` adattípus egyetlen értéke a `None`, és nincsenek rajta végezhető műveletek.
- Ez a típus és a `None` érték Python-ban általában valamilyen *érték* hiányát jelzni.
 - Ha egy változó értékét `None`-ra állítjuk, azzal azt jelezzük, hogy a változónak nincs értéke. És ez nem ugyanaz, mintha maga a változó nem létezne!
 - Ha egy listában vannak `None` értékek, azok azt jelzik, hogy az adott listaelemek értékei hiányoznak, de maguk a listaelemek megvannak.
 - Ha egy függvény a `None` értékkel tér vissza, akkor gyakorlatilag nem tér vissza semmivel, nem ad vissza értéket.
- A `NoneType` *immutable* típus, bár egyetlen értékkel és műveletek nélkül ez talán nem meglepő.

A `bool` (*logikai*) adattípus

Python VIRTUÁLIS adattípus

- Az absztrakt *logikai* adattípus megvalósítása Python-ban.
- A `bool` típus értékkészlete a `False` és `True` értékekből áll.
- A legtöbb típusú érték automatikusan konvertálható `bool` típusúvá Python-ban: az üres vagy 0 értékek `False`, az egyéb értékek `True` logikai értéké.
- A `bool` *immutable* típus.



bool (logikai) értékek műveletei

Logikai műveletek

- egyoperandusú műveletek
 not tagadás, negáció
- kétoperandusú műveletek
 and logikai és
 or logikai vagy

```
not a  
a and b  
a or b
```



bool (logikai) kifejezések

Az and és or működése Python-ban

- A logikai műveletek kiértékelése mindig a rövidített kiértékelés szerint történik, vagyis
 - Az `A or B` kifejezés rövidített kiértékelése során először kiértékelődik a `bool(A)` kifejezés, ha ennek értéke `True`, akkor a teljes kifejezés eredménye `A` lesz (a `B` tag kiértékelése elmarad); ha `bool(A)` értéke `False`, akkor a kifejezés eredménye `B` értéke lesz.
 - Az `A and B` kifejezés rövidített kiértékelése során először kiértékelődik a `bool(A)` kifejezés, ha ennek értéke `False`, akkor a teljes kifejezés eredménye `A` lesz (a `B` tag kiértékelése elmarad); ha `bool(A)` értéke `True`, akkor a kifejezés eredménye `B` értéke lesz.
- Ha `A` és `B` logikai értékek, akkor ez a logika a következőképpen egyszerűsödik:
 - Az `A or B` kifejezés rövidített kiértékelése során először kiértékelődik az `A` kifejezés, ha ennek értéke `True`, akkor a teljes kifejezés eredménye `True` lesz (a `B` tag kiértékelése elmarad); ha `A` értéke `False`, akkor a kifejezés eredménye `B` értéke lesz.
 - Az `A and B` kifejezés rövidített kiértékelése során először kiértékelődik az `A` kifejezés, ha ennek értéke `False`, akkor a teljes kifejezés eredménye `False` lesz (a `B` tag kiértékelése elmarad); ha `A` értéke `True`, akkor a kifejezés eredménye `B` értéke lesz.

bool (logikai) értékek műveletei

Relációs műveletek

- kétoperandusú műveletek

`==` azonosság

`!=` kizáró vagy (XOR)

`<` inverz implikáció tagadása

$\neg(a \leftarrow b)$

`>` implikáció tagadása

$\neg(a \rightarrow b)$

`<=` implikáció

$a \rightarrow b$

`>=` inverz implikáció

$a \leftarrow b$

```
a == b
a != b
a < b
a > b
a <= b
a >= b
```

Konverzió bool (logikai) típusra

- `bool(val)`:
 - Számok (egész, valós, komplex): a 0 értékből `False`, a nem 0 értékből `True` lesz.
 - Tárolók (sztring, lista, tuple, halmaz, dict): az üres tárolóból `False`, a nemüresből `True` lesz.
 - A `None` értékből `False` lesz.

```
>>> bool(0)
False
>>> bool(2+4j)
True
>>> bool([])
False
>>> bool({2,4})
True
>>> bool('')
False
>>> bool(None)
False
>>> bool('str')
True
```

A bytes adattípus

Python VIRTUÁLIS adattípus

- A `bytes` egy bájtok sorozatát tárolni képes *immutable* típus, felfogható úgy, mint egy pusztán egybájtos karakterek kódjait tartalmazó sztring.
- A `bytes` típus műveletei nagyon hasonlóak az `str` típus műveleteihez de a két típus közötti konverzió nem automatikus.
- A bináris (`b` prefixszel ellátott) sztring literálok `bytes` típusú értékeket jelölnek.

```
>>> b'Only ASCII characters'
b'Only ASCII characters'
>>> bytes('árvíztűrő tükörfúrógép', encoding='utf-8')
b'\xc3\xa1rv\xcc3\xadzt\xcc5\xbr\xcc5\x91 t\xcc3\xbc\xcc3\xb6rf\xcc3\xbar\xcc3\xb3g\xcc3\xa9p'
>>> b = b'\xe1rv\xedzt\xfb\xfd5_t\xfc\xfc6rf\xfar\xfg\xe9p'
>>> b.decode(encoding='iso-8859-2')
'árvíztűrő tükörfúrógép'
```

A bytearray adattípus

Python VIRTUÁLIS adattípus

- A bytearray típus a bytes típus *mutable* párja.

```
>>> ba = bytearray(b'Hello')
>>> ba[0] = ord('h')
>>> ba.decode('ascii')
'hello'
```



Tartalomjegyzék

- 
- 1 **Bemutakozás**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 **Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 **Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 **Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 **Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 **Python sztringek**
 - Algoritmus vezérlése
 - A Python nyelv utasításai
 - 7 **Vezérlési szerkezetek**
 - Python sztringek
 - Basename meghatározása
 - 8 **Python listák**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 9 **Vezérlési szerkezetek**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 10 **Python iterációk**
 - Csúszóátlag számítása
 - 11 **Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 **Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 **Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 **Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 **Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 **Python fájlkezelés**
 - Fájlkezelés
 - 17 **Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 **Python formátált IO**
 - Python formátált IO
 - 19 **Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 **Python típusok**
 - Python numerikus típusok
 - 21 **Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 **Python típusok**
 - További Python típusok
 - **Python function adattípus**
 - 23 **Absztrakt adattípusok**
 - Összetett adattípusok
 - 24 **Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 **Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 **More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A function (függvény) adattípus

Python VIRTUÁLIS adattípus

- A függvényre eddig mint algoritmusra gondoltunk.
- A Neumann-architektúra szerint az algoritmusok a számítógépben ugyanúgy (bitsorozatként) és ugyanott (memóriában) tárolódnak mint az adatok.
- A függvény azonosító (név) tulajdonképpen az algoritmus implementációját azonosítja.
- Miért ne adhatnánk ezt az algoritmust értékül egy másik azonosítónak (változónak)?

```
>>> def f(x):  
...     return x ** x  
...  
>>> variable = f  
>>> variable(3)  
27  
>>> variable(4)  
256
```

A function (függvény) adattípus műveletei

Operátorok, reláció

- műveletek
 - () függvényhívás
- összehasonlítás
 - `==` ugyanaz a függvény
 - `!=` másik függvény

```
fgv(x, y)
```

```
fgv == f
```

```
fgv != min
```



Rendezés több szempont szerint

Problémafelvetés és specifikáció

- Problémafelvetés
 - Adott hallgatók egy listája a számonkéréseken elért átlagpontszámaikkal.
 - Rendezzük a listát több szempont szerint is és minden rendezés után legyen kiíratás!
- Specifikáció
 - Az input név-adat párok sorozata, amelynek végét egy üres név jelzi.
 - Az output a név szerint növekvő, adat szerint növekvő illetve csökkenő sorrendben rendezett három lista.



Beszűrő rendezés

Problémafelvetés és specifikáció

- Problémafelvetés
 - Rendezzük egy lista elemeit!
- Specifikáció
 - Használjunk saját rendező algoritmus implementációt.
 - Az algoritmus inputja egy lista, és egy rendezési reláció.
 - Az algoritmus a megadott reláció alapján helyben rendezi a listát.



- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



2	5	3	1	4
---	---	---	---	---

- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



5

2		3	1	4
---	--	---	---	---

- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



2	5	3	1	4
---	---	---	---	---

- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



3

2	5		1	4
---	---	--	---	---

- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



3

2		5	1	4
---	--	---	---	---

- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



1

2	3	5		4
---	---	---	--	---

- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



1

2	3		5	4
---	---	--	---	---

- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



1

2		3	5	4
---	--	---	---	---

- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



1

	2	3	5	4
--	---	---	---	---

- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



4

1	2	3	5	
---	---	---	---	--

- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.



4

1	2	3		5
---	---	---	--	---

- A listát logikailag egy már rendezett és egy még rendezetlen részre osztjuk, és a rendezetlen rész első elemét beszúrjuk a rendezett elemek közé úgy, hogy azok rendezettek maradjanak.

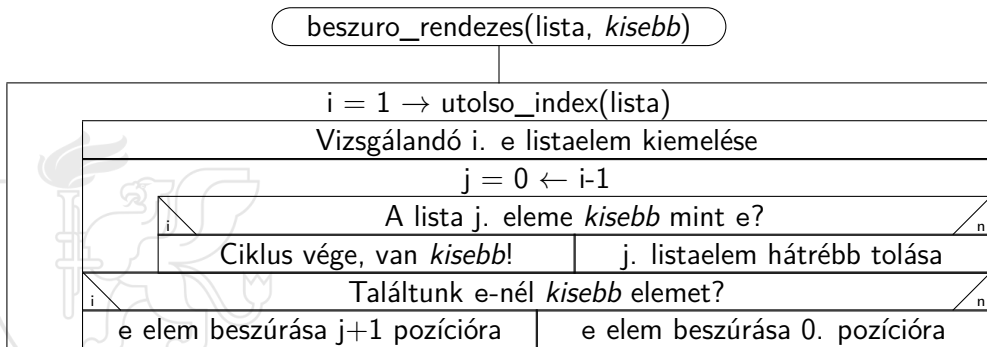


1	2	3	4	5
---	---	---	---	---

Beszúró rendezés

Algoritmustervezés – Struktogram

- A beszúró rendezés struktogramja:



Beszúró rendezés [1/3]

rendezes_manual.py [1-20]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Rendezzük névsorba illetve átlag szerint a hallgatókat!
9"""
10def beolvas():
11    n = input("Név? ")
12    if not n:
13        return None
14    p = float(input("Átlag? "))
15    return {'nev': n, 'adat': p}
16
17def kiiras(msg, l):
18    print(msg)
19    for e in l:
20        print("%s: %.2f" % (e['nev'], e['adat']))
```

Beszúró rendezés [2/3]

rendezes_manual.py [22-41]

```
22 def rend_nev_novekvo(x, y):
23     return (x['nev'] < y['nev'])
24
25 def rend_adat_novekvo(x, y):
26     return x['adat'] < y['adat']
27
28 def rend_adat_csokkeno(x, y):
29     return x['adat'] > y['adat']
30
31 def beszuro_rendezes(list_, kisebb):
32     for i in range(1, len(list_)):
33         e = list_[i]
34         for j in range(i-1, -1, -1):
35             if kisebb(list_[j], e):
36                 break
37             list_[j+1] = list_[j]
38         else:
39             list_[0] = e
40             continue
41     list_[j+1] = e
```

Beszúró rendezés [3/3]

rendezes_manual.py [43–60]

```
43 def main():
44     sor = []
45     while (e := beolvas()) is not None:
46         sor.append(e)
47         print(e)
48     kiiras("Eredeti_sorrend:", sor)
49
50     beszuro_rendezes(sor, rend_nev_novekvo)
51     kiiras("Névsor_szerint_rendezve:", sor)
52
53     beszuro_rendezes(sor, rend_adat_novekvo)
54     kiiras("Adat_szerint_rendezve:", sor)
55
56     beszuro_rendezes(sor, rend_adat_csokkeno)
57     kiiras("Adat_szerint_csökkenő_sorba_rendezve:", sor)
58
59 if __name__ == "__main__":
60     main()
```

Függvény határozott integráljának kiszámítása

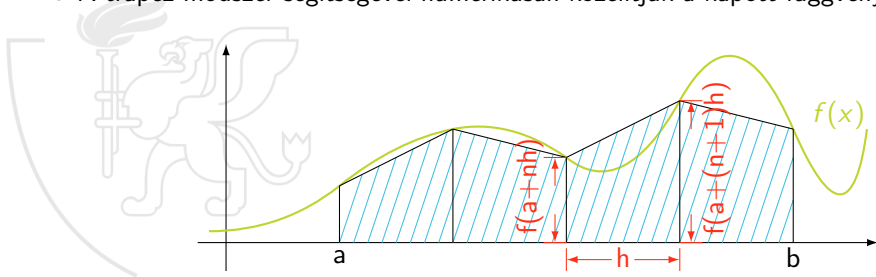
Problémafelvetés és specifikáció

- Problémafelvetés
 - Adott függvény határozott integrálját közelítsük egy adott intervallumon a felhasználó által megadott számú részre osztva az intervallumot!
- Specifikáció
 - A probléma inputja az integrálandó függvény neve, a , b valós számok, az intervallum végpontjai, n , a közelítéshez használandó részek száma.
 - Az inputot a parancssorból vegye a program.
 - Az output három valós szám: a megadott függvény értéke a és b pontokban, valamint a határozott integrál értéke.

Függvény határozott integráljának kiszámítása

Algoritmustervezés

- A fő algoritmusban:
 - Lekezeljük a parancssori paramétereket az argparse modul segítségével.
 - Kiszámítjuk a határozott integrált egy trapez függvény segítségével.
 - Kezeljük, ha hibába futunk.
- A trapez függvényben:
 - A trapéz módszer segítségével numerikusan közelítjük a kapott függvény integrálját.



Függvény határozott integráljának kiszámítása [1/3]

fgvint.py [1-21]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Függvény határozott integráljának kiszámítása.
9"""
10import argparse
11from math import sin
12from math import cos
13from math import tan
14from math import exp
15from sys import stderr
16
17def expx(x):
18    return exp(x) / x
19
20def sin2x(x):
21    return sin(2.0 * x)
```

Függvény határozott integráljának kiszámítása [2/3]

fgvint.py [23–36]

```
23 ISMERT_FUGGVENYEK = {
24     "sin": sin,
25     "tan": tan,
26     "cos": cos,
27     "exp": exp,
28     "sin2x": sin2x
29 }
30
31 def trapez(f, a, b, n):
32     h = (b - a) / n
33     area = 0.0
34     for i in range(1, n):
35         area += f(a + i * h)
36     return (area + (f(a) + f(b)) / 2.0) * h
```

Függvény határozott integráljának kiszámítása [3/3]

fgvint.py [38–62]

```
38 def main():
39     global ISMERT_FUGGVENYEK
40     parser = argparse.ArgumentParser(description = 'Függvény_határozott_integrálját_számitó_program')
41     parser.add_argument('fgv', metavar='FÜGGVÉNY', default=None, type=str,
42                         help='Az_integrálandó_függvény', choices = ISMERT_FUGGVENYEK.keys())
43     parser.add_argument('a', metavar='A', default=None, type=float,
44                         help='Az_integrál_intervallumának_alsó_végpontja')
45     parser.add_argument('b', metavar='B', default=None, type=float,
46                         help='Az_integrál_intervallumának_felső_végpontja')
47     parser.add_argument('res', metavar='N', default=None, type=int,
48                         help='Az_intervallum_felbontása')
49     args = parser.parse_args()
50
51     f = ISMERT_FUGGVENYEK[args.fgv]
52     try:
53         print("{f}({x:.3})={fx:.3}".format(f=args.fgv, x=args.a, fx=f(args.a)))
54         print("{f}({x:.3})={fx:.3}".format(f=args.fgv, x=args.b, fx=f(args.b)))
55         print("Az_{f}(x)_függvény_határozott_integrálja_a(z)_{a:.3},{b:.3}_intervallumon_{fiab:.3}.".
56               format(f=args.fgv, a=args.a, b=args.b, fiab=trapez(f, args.a, args.b, args.res)))
57     except ZeroDivisionError:
58         print("A_függvény_az_{a:.3},{b:.3}_intervallum_valamely_pontjában_nem_értelmezett!".
59               format(a=args.a, b=args.b), file=stderr)
60
61 if __name__ == "__main__":
62     main()
```